Lecture 5 — Offline Techniques

> World Finals Level · Prerequisites: *week-19* (DSU), *week-24* (Fenwick trees), *week-34* (sqrt decomposition, Mo's algorithm), *nationals-02* (persistent / merge-sort trees) · Estimated time: 2 × 90 minutes

1. Overview

A problem is **offline** when all queries are known before you have to answer any of them. Then you are free to reorder queries, process them in batches, or replay the update sequence several times. Many problems that would need heavy online data structures (2D trees, fully dynamic connectivity, persistent structures) become short and fast offline.

This lecture covers four techniques:

Technique	Core idea	Typical cost
CDQ divide and conquer	split on one dimension, so only "left contributes to right" pairs remain	$O(n \log^2 n)$
Parallel binary search	binary search for every query at once, sharing one replay of the updates per round	$O((M + Q) \log M) \times$ update cost
Segment tree over time	each item lives on a time interval; insert it into $O(\log T)$ nodes and use an undoable structure	$O(T \log T) \times$ undo cost
Mo's algorithm with updates	add a time pointer to Mo's (l, r) pointers	O(n^{5/3})

All four reference programs in Section 7 were stress-tested against brute force on random inputs.

A warning sign to read for in statements: if queries are **encoded with the previous answer** (`x ← x xor lastans`), the setters have deliberately forced an online solution, and none of these techniques apply.

2. CDQ divide and conquer

2.1 The pattern

CDQ (named after Chen Danqi, who popularised it in Chinese olympiad training) applies when the answer is a sum of contributions over pairs `(j, i)` with `j` before `i` in some order, plus further conditions. Recursively:

```
solve(l, r):
    solve(l, m); solve(m, r)
    add the contributions of every j in [l, m) to every i in [m, r)
```

Every pair with `j < i` is handled exactly once, in the call where they are split. In that cross step, the condition "j before i" holds automatically, so one dimension of the problem disappears. If the cross step costs O(len log len), the total is **O(n log² n)**.

2.2 3D dominance counting

For each point i, count the points j ≠ i with `a_j ≤ a_i`, `b_j ≤ b_i`, `c_j ≤ c_i`.

- **Dimension a** is handled by sorting by `(a, b, c)` and splitting.
- **Dimension b** is handled by the order inside the cross step. The recursion leaves each half sorted by b, merge-sort style, so we sweep the right half in increasing b and insert left items with `b ≤` the current b.
- **Dimension c** is handled by a Fenwick tree indexed by c.

Duplicates. The cross step only counts left → right. With lexicographic sorting, a right item can dominate a left item only if they are identical. Identical triples are merged into one item with multiplicity `cnt` *before* the recursion, and each copy of an item dominates `res + cnt − 1` others: its result plus the other copies of itself. Without the merge, two identical points split across halves would count each other asymmetrically.

Undo, don't clear. After the cross step, remove exactly the BIT entries that were added. Clearing the whole BIT costs O(K) per call and makes the algorithm quadratic.

2.3 Dynamic to static: time as a dimension

Suppose the operations "add w at point (x, y)" and "sum over a rectangle" are interleaved. Give each operation its index t. A query at time t needs the updates with `t′ < t`, `x′ ≤ X`, `y′ ≤ Y` (after splitting the rectangle into four prefix queries with signs). That is a 3D dominance problem with time as the first coordinate, and it is already sorted. CDQ gives O(n log² n) with linear memory and no 2D structure.

The same trick turns "data structure that supports inserts but not deletes, plus interleaved queries" into a sequence of static problems. For example, a convex hull trick with arbitrary insertion order becomes a static hull built in each cross step.

2.4 CDQ for DP and online convolution

For DP of the form `dp[i] = combine over j < i with cond(j, i) of dp[j]` (for example, the longest chain in 3D, or 2D LIS with an extra constraint), the right half needs the **final** values of the left half. The order must be:

```
solve(l, m); cross(l, m, r); solve(m, r)
```

The cross step must leave the right half in the order its own recursion expects. Sort copies, or restore the order afterwards.

The same "left, cross, right" schedule computes **online (relaxed) convolutions** `f_i = Σ_{j<i} f_j g_{i−j}`, where f depends on itself. In the cross step, convolve `f[l..m)` with `g[0..r−l)` by NTT and add the result to `f[m..r)`. That is O(n log² n), and it is the standard way to compute exp of a power series or count recursive structures when Newton's method (Lecture 2) is inconvenient.

3. Parallel binary search

3.1 The pattern

There are M updates applied in order and Q queries. Query i asks for the smallest k such that a predicate `P_i` holds on the state after updates 1..k, and `P_i` is monotone in k. One query alone is a binary search whose probes each replay a prefix of updates: O(M log M). Doing this independently for Q queries is far too slow.

Batching. Keep `[lo_i, hi_i]` for every unresolved query. In each round:

1. Put every unresolved query into the bucket for its midpoint `mid_i = ⌊(lo_i + hi_i)/2⌋`.
2. Reset the state and apply updates 1..M in order. Right after applying update k, evaluate `P_i` for every query in bucket k, and set `hi_i = k` or `lo_i = k + 1`.

Each round halves every interval, so there are ⌈log₂ M⌉ + 1 rounds. Each round costs O(M · update + Q · check). With a DSU this is O((M + Q) log M · α(n)).

3.2 Requirements and variants

- Each predicate must be monotone in k.
- You must be able to replay the updates from scratch each round, or support undo.
- For queries that may never be satisfied, search on `[1, M + 1]` and treat M + 1 as "never". The final round then needs no check at M + 1.
- **POI 2011 *Meteors*** (SPOJ METEORS): each member state owns some sectors of a circular orbit, and meteor showers add values to circular ranges. For each state, find the first shower after which its total reaches its target. Updates are range additions (a BIT with range add and point query), and a check sums the BIT over the state's sectors. The total work is linear in the number of sectors per round.
- **AtCoder AGC002 D *Stamp Rally*** (reference code): the state is a DSU over edges added in order, and the check is "do the components of x and y contain ≥ z vertices together?"

3.3 Relation to other techniques

Parallel binary search can often be replaced by a persistent data structure (binary search on versions) or by a Kruskal reconstruction tree for DSU problems. It is usually the simplest to write, and it uses only O(n + Q) memory.

4. Offline dynamic connectivity

4.1 Segment tree over time

The operations are add edge, remove edge, and a query (are u and v connected? how many components?). DSU supports unions but not deletions. Offline, every edge occurrence is alive on a time interval `[t_add, t_remove)`, or `[t_add, T)` if it is never removed. Build a segment tree over the time indices `[0, T)` and insert each interval into the O(log T) canonical nodes that cover it. Then DFS the segment tree:

- on entering a node, union all of its edges;
- at a leaf t, the DSU contains exactly the edges alive at time t, so answer the query;
- on leaving a node, roll back the unions made there.

4.2 Rollback DSU

- **No path compression.** Compression rewrites many pointers and cannot be undone cheaply, and its amortised bound does not survive rollbacks anyway.
- **Union by size (or rank).** This alone bounds the tree depth by $\log_2 n$, so `find` is O(log n).
- **Record every union**, including no-ops where both ends are already connected, so that "roll back as many times as I united" is always correct.

Total cost: O(T log T) unions and rollbacks, each O(log n) because of the finds: **O(T log T log n)**. For $T = 3 \cdot 10^5$ this runs comfortably in time.

4.3 Variations

- **Bipartiteness under insertions and deletions** (Codeforces 813F). Store the parity of each vertex relative to its parent. An edge whose endpoints are in the same component with equal parity creates an odd cycle. Every leaf in the current segment-tree subtree is then non-bipartite, and you can stop descending (but still roll back).
- **Aggregates per component** (sum of vertex weights, size): keep them at the roots and restore them on rollback (Library Checker *Dynamic Graph Vertex Add Component Sum*).
- **Multi-edges**: pair each removal with the most recent unmatched insertion of the same (unordered) pair.

- **Other undoable structures.** Anything with cheap undo works: linear bases (xor bases) for "maximum xor path" style queries, convex hull insertions with a stack, knapsack arrays (O(capacity) per item).

4.4 Online alternatives

Fully dynamic connectivity online is possible in $O(\log^2 n)$ amortised (Holm, de Lichtenberg, Thorup), but it is long and essentially never required in contests. A common semi-offline alternative is a link-cut tree (Lecture 4) that maintains a spanning forest **maximising deletion times**. When a new edge closes a cycle, remove the cycle edge that is deleted earliest. This requires knowing deletion times, so it is still offline.

5. Mo's algorithm with updates

5.1 The idea

Week 34's Mo's algorithm answers range queries `(l, r)` by sorting them so the two pointers move $O((n + q)\sqrt{n})$ in total. With point assignments interleaved, give each query a third coordinate `t`, the number of updates before it. Maintain a current state `(L, R, t)`:

- moving t forward applies update t+1: if its position lies in `[L, R]`, remove the old value from the statistic and add the new one; always write the new value into the array;
- moving t backward undoes it symmetrically.

Store each update's **old value** when reading the input by simulating the array once. Then applying and undoing are both O(1).

5.2 Ordering and block size

Sort queries by `(⌊l/B⌋, ⌊r/B⌋, t)`. Count pointer moves:

- l and r move O(B) per query, plus O(n) per block change: $O(Q \cdot B + n^2/B)$;
- t moves O(U) for each pair of (l-block, r-block): $O((n/B)^2 \cdot U)$.

Balancing `Q·B` against `n²U/B²` gives `B ≈ (n²U/Q)^{1/3}`, which is `n^{2/3}` when U and Q are both of order n. The total is **`O(n^{5/3})`**. For $n = q = 1.3 \cdot 10^5$ this is about $3 \cdot 10^8$ cheap pointer operations. The reference code runs Luogu P1903's maximum size in about 0.8 s.

Alternating the direction of r (and t) between blocks ("odd-even" sorting) is a free constant-factor improvement and is used in the reference code.

5.3 When to use it

Use it when the statistic supports O(1) add/remove of a single element (distinct counts, frequency-of-frequency, mex via counting plus sqrt blocks as in Codeforces 940F), the problem has point updates, and n ≈ 10^5. For n ≥ $3 \cdot 10^5$, n^{5/3} is too slow: look for a CDQ or segment-tree-over-time formulation instead.

6. Choosing among them

Situation	First choice
counting pairs under three order constraints	CDQ
interleaved inserts + queries, static structure known	CDQ over time
"when does each X first reach its target?"	parallel binary search
deletions on a structure that only supports insert + undo	segment tree over time
range statistic + point updates, O(1) add/remove, n ≈ 10^5	Mo with updates

These overlap. 3D dominance can also be solved with a BIT of sorted vectors (nationals-02 style), and *Stamp Rally* has a Kruskal-reconstruction-tree solution. In a contest, pick the version whose code you have already tested.

7. Reference implementations

7.1 CDQ: 3D partial order (Luogu P3810)

Input: `n K`, then n triples with coordinates in `[1, K]`. Output: for d = 0..n−1, the number of points dominating exactly d others.

```
#include <bits/stdc++.h>
using namespace std;

// Luogu P3810 (3D partial order): f(i) = #{j != i : a_j <= a_i, b_j <= b_i, c_j <= c_i}.
// Output, for d = 0..n-1, how many i have f(i) = d.  CDQ divide & conquer, O(n log^2 n).
struct Item { int a, b, c, cnt, res; };

struct BIT {
    int n; vector<int> t;
    BIT(int n) : n(n), t(n + 1, 0) {}
    void add(int i, int v) { for (; i <= n; i += i & -i) t[i] += v; }
    int sum(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; }
};

vector<Item> it, tmp;
BIT* bit;
```

```
// On entry it[l..r) is sorted by a (ties already merged). On exit it is sorted by b.
void cdq(int l, int r) {
    if (r - l <= 1) return;
    int m = (l + r) / 2;
    cdq(l, m);
    cdq(m, r);
    // Every left item has a smaller a than every right item. Both halves are sorted by
b.
    int i = l, k = l;
    for (int j = m; j < r; j++) {
        while (i < m && it[i].b <= it[j].b) { bit->add(it[i].c, it[i].cnt); tmp[k++] =
it[i++]; }
        it[j].res += bit->sum(it[j].c);
        tmp[k++] = it[j];
    }
    for (int x = l; x < i; x++) bit->add(it[x].c, -it[x].cnt);   // undo exactly what we
added
    while (i < m) tmp[k++] = it[i++];
    for (int x = l; x < r; x++) it[x] = tmp[x];                  // merge by b
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, K;
    cin >> n >> K;
    vector<array<int,3>> p(n);
    for (auto& x : p) cin >> x[0] >> x[1] >> x[2];
    sort(p.begin(), p.end());
    for (int i = 0; i < n; i++) {                                 // merge identical
triples
        if (it.empty() || it.back().a != p[i][0] || it.back().b != p[i][1] || it.back().c
!= p[i][2])
            it.push_back({p[i][0], p[i][1], p[i][2], 0, 0});
        it.back().cnt++;
    }
    tmp.resize(it.size());
    BIT b(K); bit = &b;
    cdq(0, it.size());
    vector<int> ans(n, 0);
    for (auto& x : it) ans[x.res + x.cnt - 1] += x.cnt;           // equal items dominate
each other
    for (int d = 0; d < n; d++) cout << ans[d] << "\n";
}
```

7.2 Parallel binary search (AtCoder AGC002 D *Stamp Rally*)

```
#include <bits/stdc++.h>
using namespace std;

// AtCoder AGC002 D "Stamp Rally": edges are added in order 1..M. Query (x, y, z): the
smallest k
// such that, using edges 1..k, the components of x and y together contain at least z
vertices.
```

```cpp
// Parallel binary search: O((M + Q) log M * alpha).
struct DSU {
    vector<int> p, s;
    DSU(int n) : p(n + 1), s(n + 1, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { while (p[x] != x) x = p[x] = p[p[x]]; return x; }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return;
        if (s[a] < s[b]) swap(a, b);
        p[b] = a; s[a] += s[b];
    }
    int reach(int x, int y) { x = find(x); y = find(y); return x == y ? s[x] : s[x] +
s[y]; }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int,int>> e(m);
    for (auto& [a, b] : e) cin >> a >> b;
    int q; cin >> q;
    vector<int> x(q), y(q), z(q);
    for (int i = 0; i < q; i++) cin >> x[i] >> y[i] >> z[i];

    vector<int> lo(q, 1), hi(q, m);              // invariant: answer in [lo, hi]
    vector<vector<int>> bucket(m + 1);
    while (true) {
        bool any = false;
        for (auto& b : bucket) b.clear();
        for (int i = 0; i < q; i++)
            if (lo[i] < hi[i]) bucket[(lo[i] + hi[i]) / 2].push_back(i), any = true;
        if (!any) break;
        DSU d(n);                                 // one sweep over time answers all
midpoints
        for (int k = 1; k <= m; k++) {
            d.unite(e[k - 1].first, e[k - 1].second);
            for (int i : bucket[k]) {
                if (d.reach(x[i], y[i]) >= z[i]) hi[i] = k;
                else lo[i] = k + 1;
            }
        }
    }
    for (int i = 0; i < q; i++) cout << lo[i] << "\n";
}
```

7.3 Offline dynamic connectivity

```cpp
#include <bits/stdc++.h>
using namespace std;

// Offline dynamic connectivity (format of Codeforces Gym 100551A "Connect and
Disconnect";
```

```cpp
// that judge uses files connect.in / connect.out). Operations: "+ u v" add edge, "- u v" remove
// edge, "?" print the number of connected components. Segment tree over time + rollback DSU,
// O(k log k log n).
struct RollbackDSU {
    vector<int> p, s; vector<pair<int,int>> hist; int comps;
    RollbackDSU(int n) : p(n + 1), s(n + 1, 1), comps(n) { iota(p.begin(), p.end(), 0); }
    int find(int x) { while (p[x] != x) x = p[x]; return x; }     // NO path compression
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) { hist.push_back({-1, -1}); return; }          // record a no-op too
        if (s[a] < s[b]) swap(a, b);                               // union by size: depth O(log n)
        p[b] = a; s[a] += s[b]; comps--;
        hist.push_back({a, b});
    }
    void rollback() {
        auto [a, b] = hist.back(); hist.pop_back();
        if (a == -1) return;
        p[b] = b; s[a] -= s[b]; comps++;
    }
};

int T;
vector<vector<pair<int,int>>> seg;
vector<char> isQuery;
vector<int> answer;

void insertEdge(int node, int l, int r, int ql, int qr, pair<int,int> e) {   // alive on [ql, qr)
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) { seg[node].push_back(e); return; }
    int m = (l + r) / 2;
    insertEdge(2 * node, l, m, ql, qr, e);
    insertEdge(2 * node + 1, m, r, ql, qr, e);
}

void solve(int node, int l, int r, RollbackDSU& d) {
    for (auto [u, v] : seg[node]) d.unite(u, v);
    if (r - l == 1) {
        if (isQuery[l]) answer[l] = d.comps;
    } else {
        int m = (l + r) / 2;
        solve(2 * node, l, m, d);
        solve(2 * node + 1, m, r, d);
    }
    for (size_t i = 0; i < seg[node].size(); i++) d.rollback();   // undo in reverse order
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n;
    cin >> n >> T;
```

```
    if (T == 0) return 0;
    seg.assign(4 * T, {});
    isQuery.assign(T, 0);
    answer.assign(T, 0);
    map<pair<int,int>, vector<int>> open;          // edge -> start times (multi-edges
allowed)
    for (int i = 0; i < T; i++) {
        char c; cin >> c;
        if (c == '?') { isQuery[i] = 1; continue; }
        int u, v; cin >> u >> v;
        if (u > v) swap(u, v);
        if (c == '+') open[{u, v}].push_back(i);
        else {
            auto& st = open[{u, v}];
            insertEdge(1, 0, T, st.back(), i, {u, v});
            st.pop_back();
        }
    }
    for (auto& [e, st] : open)
        for (int s : st) insertEdge(1, 0, T, s, T, e);  // never removed: alive until the
end
    RollbackDSU d(n);
    solve(1, 0, T, d);
    for (int i = 0; i < T; i++) if (isQuery[i]) cout << answer[i] << "\n";
}
```

7.4 Mo's algorithm with updates (Luogu P1903)

```
#include <bits/stdc++.h>
using namespace std;

// Luogu P1903 (数颜色): "Q l r" = number of distinct values in a[l..r], "R p c" = set
a[p] = c.
// Mo's algorithm with updates: O(n^{5/3}) for n, q of the same order.
struct Query { int l, r, t, id; };
struct Update { int pos, oldV, newV; };

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<Query> qs; vector<Update> ups;
    vector<int> cur = a;                           // array state as updates are read
    int maxV = *max_element(a.begin(), a.end());
    for (int i = 0; i < m; i++) {
        char c; int x, y;
        cin >> c >> x >> y;
        if (c == 'Q') qs.push_back({x, y, (int)ups.size(), (int)qs.size()});
        else { ups.push_back({x, cur[x], y}); cur[x] = y; maxV = max(maxV, y); }
    }
    // Block size ~ (n^2 U / Q)^{1/3}; equals n^{2/3} when U and Q are both ~ n.
```

```
    double U = max<size_t>(ups.size(), 1), Q = max<size_t>(qs.size(), 1);
    int B = max(1, (int)cbrt((double)n * n * U / Q));
    sort(qs.begin(), qs.end(), [&](const Query& p, const Query& q) {
        if (p.l / B != q.l / B) return p.l < q.l;
        if (p.r / B != q.r / B) return ((p.l / B) & 1) ? p.r > q.r : p.r < q.r;
        return ((p.r / B) & 1) ? p.t > q.t : p.t < q.t;
    });
    vector<int> cnt(maxV + 1, 0), res(qs.size());
    int L = 1, R = 0, t = 0, distinct = 0;
    auto add = [&](int v) { if (cnt[v]++ == 0) distinct++; };
    auto del = [&](int v) { if (--cnt[v] == 0) distinct--; };
    auto apply = [&](int i, bool forward) {       // move time across update i
        const Update& u = ups[i];
        int from = forward ? u.oldV : u.newV, to = forward ? u.newV : u.oldV;
        if (L <= u.pos && u.pos <= R) { del(from); add(to); }
        a[u.pos] = to;
    };
    for (const Query& q : qs) {
        while (t < q.t) apply(t++, true);
        while (t > q.t) apply(--t, false);
        while (L > q.l) add(a[--L]);
        while (R < q.r) add(a[++R]);
        while (L < q.l) del(a[L++]);
        while (R > q.r) del(a[R--]);
        res[q.id] = distinct;
    }
    for (int x : res) cout << x << "\n";
}
```

8. Common mistakes

- **CDQ, full BIT clears.** Clearing with `fill` in every call is O(K) per call. Undo exactly what was inserted.
- **CDQ, equal keys.** Unmerged duplicates and ties in the split coordinate give asymmetric counts. Make sure "right contributes to left" pairs are impossible.
- **CDQ for DP in the wrong order.** Doing `solve(m, r)` before the cross step uses unfinished values.
- **CDQ with recursion on sub-arrays plus an extra sort per level.** That adds a log factor. Merge, don't sort.
- **PBS without a fresh state per round**, or off-by-one confusion between "state after update k" and "before update k".
- **PBS on non-monotone predicates.** The search silently returns garbage. Double-check monotonicity (for example, "exactly z vertices" is not monotone, but "at least z" is).
- **Rollback DSU with path compression**, or a `unite` that pushes no record on no-ops, which desynchronises the rollback count.
- **Segment tree over time**: forgetting edges that are never removed, or using a closed interval `[t_add, t_remove]` so the edge is still alive at the removal step.

- **Mo with updates**: forgetting to check `L ≤ pos ≤ R` before adjusting the statistic, or using √n blocks, which gives an O(n²)-ish time component.

9. Practice problems

1. Luogu P3810 — 【模板】三维偏序（陌上花开）: https://www.luogu.com.cn/problem/P3810
2. Codeforces 848C — *Goodbye Souvenir* (CDQ over time): https://codeforces.com/problemset/problem/848/C
3. Codeforces 1093E — *Intersection of Permutations* (CDQ or BIT of BITs): https://codeforces.com/problemset/problem/1093/E
4. AtCoder AGC002 D — *Stamp Rally* (parallel binary search): https://atcoder.jp/contests/agc002/tasks/agc002_d
5. SPOJ METEORS — *Meteors* (POI 2011): https://www.spoj.com/problems/METEORS/
6. Codeforces Gym 100551A — *Connect and Disconnect* (segment tree over time): https://codeforces.com/gym/100551/problem/A
7. Codeforces 813F — *Bipartite Checking*: https://codeforces.com/problemset/problem/813/F
8. Library Checker — *Dynamic Graph Vertex Add Component Sum*: https://judge.yosupo.jp/problem/dynamic_graph_vertex_add_component_sum
9. Luogu P1903 — [国家集训队] 数颜色 / 维护队列 (Mo with updates): https://www.luogu.com.cn/problem/P1903
10. Codeforces 940F — *Machine Learning* (Mo with updates, mex of counts): https://codeforces.com/problemset/problem/940/F

10. Further reading

- cp-algorithms, "Deleting from a data structure in O(T(n) log n)" (segment tree over time): https://cp-algorithms.com/data_structures/deleting_in_log_n.html
- cp-algorithms, "Sqrt Decomposition" (Mo's algorithm and variants): https://cp-algorithms.com/data_structures/sqrt_decomposition.html
- Codeforces blog "Parallel Binary Search [tutorial]" (search for it on codeforces.com), which also covers *Meteors*.
- The official Codeforces editorial of problem 940F, for the n^{2/3} block analysis applied to a real problem.
- J. Holm, K. de Lichtenberg, M. Thorup, "Poly-logarithmic deterministic fully-dynamic algorithms for connectivity…", JACM 48(4), 2001 — the online alternative, for context.